

Data Warehouse Macrofinanciero
Proyecto Guiado A→Z

Francisco Ramírez

12 de junio de 2026

Índice general

1. Hito 1: Creación del Proyecto	6
1.1. Objetivo	6
1.2. Resultado esperado	6
1.3. Paso 1: Crear la carpeta del proyecto	7
1.4. Paso 2: Abrir el proyecto en VS Code	7
1.5. Paso 3: Inicializar Git	7
1.6. Paso 4: Crear la estructura de carpetas	8
1.7. Paso 5: Crear los archivos iniciales	8
1.8. Paso 6: Configurar el archivo .gitignore	9
1.9. Paso 7: Crear el README inicial	9
1.10. Paso 8: Crear el entorno virtual	10
1.11. Paso 9: Instalar dependencias mínimas	10
1.12. Paso 10: Realizar el primer commit	10
1.13. Checklist de validación	11
1.14. Entregable	11
1.15. Próximo hito	12
2. Hito 2: Instalación y Configuración de PostgreSQL	13
2.1. Objetivo	13
2.2. Resultado esperado	13
2.3. Paso 1: Verificar instalación de PostgreSQL	14
2.4. Paso 2: Iniciar PostgreSQL	14
2.5. Paso 3: Conectarse al servidor	14
2.6. Paso 4: Crear la base de datos	15
2.7. Paso 5: Conectarse a la nueva base	15
2.8. Paso 6: Crear archivo .env	15
2.9. Paso 7: Actualizar .env.example	16
2.10. Paso 8: Crear módulo de conexión	16
2.11. Paso 9: Probar la conexión	17

2.12. Paso 10: Realizar una consulta desde Python	17
2.13. Paso 11: Instalar extensiones de VS Code	17
2.14. Paso 12: Crear primer script SQL	18
2.15. Checklist de validación	18
2.16. Entregable	18
2.17. Próximo hito	19
3. Hito 2: Configuración de DuckDB como Base de Datos Analítica Local	20
3.1. Objetivo	20
3.2. Resultado esperado	21
3.3. ¿Por qué DuckDB?	21
3.4. Arquitectura del hito	21
3.5. Paso 1: Actualizar la estructura del proyecto	22
3.6. Paso 2: Instalar DuckDB	22
3.7. Paso 3: Verificar instalación	23
3.8. Paso 4: Crear archivo de configuración	23
3.9. Paso 5: Actualizar .env.example	23
3.10. Paso 6: Crear módulo de conexión	24
3.11. Paso 7: Ejecutar prueba de conexión	25
3.12. Paso 8: Verificar archivo de base de datos	25
3.13. Paso 9: Crear primer script SQL	25
3.14. Paso 10: Ejecutar SQL desde Python	26
3.15. Paso 11: Instalar extensión recomendada en VS Code	26
3.16. Paso 12: Crear consulta de prueba	27
3.17. Diferencias importantes frente a PostgreSQL	27
3.18. Buenas prácticas	28
3.19. Checklist de validación	28
3.20. Entregable	29
3.21. Próximo hito	29
4. Hito 3: Diseño de las Fuentes de Datos e Ingesta Inicial	30
4.1. Objetivo	30
4.2. ¿Por qué comenzamos con archivos CSV?	30
4.3. Arquitectura de la ingesta	31
4.4. Fuentes utilizadas	32
4.4.1. Fuente 1: Tipo de Cambio	32
4.4.2. Fuente 2: Balances Bancarios	32
4.5. Paso 1: Crear estructura de almacenamiento	33
4.6. Paso 2: Crear archivo tipo_cambio.csv	33

4.7. Paso 3: Crear archivo balances.csv	34
4.8. Paso 4: Crear script de exploración	34
4.9. Paso 5: Ejecutar exploración	35
4.10. Paso 6: Verificar estructura	35
4.11. Paso 7: Validaciones básicas	35
4.12. Paso 8: Crear capa processed	36
4.13. Paso 9: Guardar datos procesados	36
4.14. Paso 10: Documentar las fuentes	36
4.15. Entregables	37
4.16. Checklist de validación	37
4.17. Próximo hito	38
5. Hito 4: Diseño del Modelo Dimensional	39
5.1. Objetivo	39
5.2. Resultado esperado	39
5.3. Paso 1: Entender qué es un modelo dimensional	40
5.4. Paso 2: Identificar los procesos de negocio	40
5.5. Paso 3: Definir el grano	40
5.5.1. Grano de fact_tipo_cambio	41
5.5.2. Grano de fact_balance_mensual	41
5.6. Paso 4: Definir dimensiones	41
5.6.1. Dimensión tiempo	41
5.6.2. Dimensión moneda	42
5.6.3. Dimensión entidad financiera	42
5.7. Paso 5: Definir tablas de hechos	42
5.7.1. Hecho tipo de cambio	42
5.7.2. Hecho balance mensual	43
5.8. Paso 6: Definir claves	43
5.8.1. Clave de negocio	43
5.8.2. Clave subrogada	44
5.9. Paso 7: Diseñar el esquema estrella	44
5.10. Paso 8: Crear documento del modelo	45
5.11. Paso 9: Crear tabla de definición del modelo	46
5.12. Paso 10: Definir indicadores derivados	46
5.13. Paso 11: Validar el diseño	47
5.14. Checklist de validación	47
5.15. Entregable	48
5.16. Próximo hito	48

6. Hito 5: Construcción de la Capa Staging (Bronze)	49
6.1. Objetivo	49
6.2. Resultado esperado	49
6.3. ¿Qué es la capa Staging?	50
6.4. Arquitectura del Data Warehouse	50
6.5. Paso 1: Crear carpeta para scripts SQL	51
6.6. Paso 2: Crear los schemas	51
6.7. Paso 3: Ejecutar el script	51
6.8. Paso 4: Crear script de tablas staging	52
6.9. Paso 5: Diseñar tabla tipo_cambio_raw	52
6.10. ¿Por qué usamos TEXT?	52
6.11. Paso 6: Diseñar tabla balance_bancario_raw	53
6.12. Paso 7: Ejecutar el DDL	54
6.13. Paso 8: Verificar tablas creadas	54
6.14. Paso 9: Inspeccionar estructura	54
6.15. Paso 10: Crear script de prueba	54
6.16. Paso 11: Documentar la capa staging	55
6.17. Paso 12: Crear consulta de monitoreo	56
6.18. Buenas prácticas	56
6.19. Checklist de validación	56
6.20. Entregable	57
6.21. Próximo hito	57
7. Hito 5: Construcción de la Capa Staging en DuckDB	58
7.1. Objetivo	58
7.2. Resultado esperado	58
7.3. Paso 1: Crear script de schemas	59
7.4. Paso 2: Crear script para ejecutar SQL	59
7.5. Paso 3: Ejecutar creación de schemas	60
7.6. Paso 4: Verificar schemas	60
7.7. Paso 5: Crear script de tablas staging	61
7.8. Paso 6: Crear tabla tipo_cambio_raw	61
7.9. Paso 7: Crear tabla balance_bancario_raw	62
7.10. Nota sobre tipos de datos	62
7.11. Paso 8: Ejecutar DDL de staging	63
7.12. Paso 9: Verificar tablas creadas	63
7.13. Paso 10: Crear script general para inicializar la base	64
7.14. Paso 11: Ejecutar inicialización completa	65
7.15. Paso 12: Crear consulta de monitoreo	65
7.16. Buenas prácticas	65

7.17. Checklist de validación	66
7.18. Entregable	66
7.19. Próximo hito	67
8. Hito 6: Construcción del Primer Proceso ETL	68
8.1. Objetivo	68
8.2. Resultado esperado	69
8.3. Arquitectura del proceso	69
8.4. Paso 1: Verificar archivos fuente	69
8.5. Paso 2: Crear módulo de conexión	70
8.6. Paso 3: Crear script de carga	70
8.7. Paso 4: Importar librerías	70
8.8. Paso 5: Leer tipo de cambio	71
8.9. Paso 6: Leer balances	71
8.10. Paso 7: Agregar metadatos	71
8.11. Paso 8: Limpiar staging antes de cargar	72
8.12. Paso 9: Ejecutar truncado desde Python	72
8.13. Paso 10: Cargar tipo de cambio	72
8.14. Paso 11: Cargar balances	73
8.15. Paso 12: Confirmar carga	73
8.16. Versión completa del script	73
8.17. Paso 13: Ejecutar ETL	75
8.18. Paso 14: Verificar en PostgreSQL	75
8.19. Paso 15: Inspeccionar datos	75
8.20. Paso 16: Crear script de validación	76
8.21. Paso 17: Crear orquestador	76
8.22. Buenas prácticas	77
8.23. Checklist de validación	77
8.24. Entregable	77
8.25. Próximo hito	78

Capítulo 1

Hito 1: Creación del Proyecto

1.1. Objetivo

El objetivo de este hito es construir la estructura inicial del proyecto y preparar el entorno de desarrollo que utilizaremos durante todo el manual.

Al finalizar este hito el estudiante tendrá:

- Un repositorio Git inicializado.
- Una estructura de carpetas profesional.
- Un entorno virtual de Python.
- Dependencias básicas instaladas.
- El primer commit del proyecto.

1.2. Resultado esperado

Al finalizar este hito, la estructura del proyecto deberá lucir de la siguiente forma:

```
dwh_macrofinanciero_rd/  
  
+-- data/  
|   +-- raw/  
|   \-- processed/  
  
+-- sql/
```

```
+++ etl/  
+++ app/  
+++ docs/  
+++ tests/  
  
+++ README.md  
+++ requirements.txt  
+++ .gitignore
```

1.3. Paso 1: Crear la carpeta del proyecto

Abrir PowerShell y navegar al directorio donde se almacenará el proyecto.
Por ejemplo:

```
cd C:\Users\t490\Documents\MacroPol
```

Crear la carpeta principal:

```
mkdir dwh_macrofinanciero_rd  
cd dwh_macrofinanciero_rd
```

Verificar la ubicación actual:

```
pwd
```

1.4. Paso 2: Abrir el proyecto en VS Code

Desde la terminal:

```
code .
```

VS Code deberá abrir la carpeta recién creada.

1.5. Paso 3: Inicializar Git

Abrir la terminal integrada de VS Code y ejecutar:

```
git init
```

Salida esperada:

```
Initialized empty Git repository
```

Verificar el estado:

```
git status
```


1.6. Paso 4: Crear la estructura de carpetas

Crear las carpetas principales:

```
mkdir data
mkdir sql
mkdir etl
mkdir app
mkdir docs
mkdir tests
```

Crear las subcarpetas para almacenamiento de datos:

```
mkdir data\raw
mkdir data\processed
```

1.7. Paso 5: Crear los archivos iniciales

Crear los siguientes archivos:

- README.md
- requirements.txt
- .gitignore
- .env.example

La estructura del proyecto deberá verse así:

```
dwh_macrofinanciero_rd/
|
+-- data/
+-- sql/
+-- etl/
+-- app/
+-- docs/
+-- tests/
|
+-- README.md
+-- requirements.txt
+-- .gitignore
+-- .env.example
```

1.8. Paso 6: Configurar el archivo .gitignore

Crear el archivo:

```
.gitignore
```

con el siguiente contenido:

```
.venv/  
.env  
  
**pycache**/  
  
*.pyc  
*.pyo  
  
.vscode/  
  
data/processed/  
  
.DS_Store  
Thumbs.db
```

1.9. Paso 7: Crear el README inicial

Archivo:

```
README.md
```

Contenido sugerido:

```
# Data Warehouse Macrofinanciero RD  
  
Proyecto practico para construir una plataforma  
de inteligencia macrofinanciera utilizando:  
  
* PostgreSQL  
* Python  
* Streamlit  
* GitHub
```

Autor: Francisco Ramirez

1.10. Paso 8: Crear el entorno virtual

Crear el entorno virtual:

```
python -m venv .venv
```

Activarlo:

Windows:

```
.venv\Scripts\activate
```

Linux/Mac:

```
source .venv/bin/activate
```

La terminal deberá mostrar:

```
(.venv)
```

1.11. Paso 9: Instalar dependencias mínimas

Instalar los paquetes necesarios:

```
pip install pandas  
pip install sqlalchemy  
pip install psycopg2-binary  
pip install python-dotenv
```

Guardar las dependencias instaladas:

```
pip freeze > requirements.txt
```

1.12. Paso 10: Realizar el primer commit

Agregar todos los archivos al repositorio:

```
git add .
```

Verificar:

```
git status
```

Crear el primer commit:

```
git commit -m "Initial_project_structure"
```

Consultar el historial:

```
git log --oneline
```

Salida esperada:

```
3f2a9ab Initial project structure
```

1.13. Checklist de validación

Antes de continuar al siguiente hito, verificar:

- ☐ VS Code instalado y funcionando.
- ☐ Proyecto creado.
- ☐ Git inicializado.
- ☐ Estructura de carpetas creada.
- ☐ Entorno virtual activo.
- ☐ Dependencias instaladas.
- ☐ Primer commit realizado.

1.14. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Un proyecto abierto en VS Code.
- Un repositorio Git funcional.
- Un entorno virtual configurado.
- Una estructura base lista para desarrollar el Data Warehouse.

1.15. Próximo hito

En el Hito 2 construiremos la infraestructura de base de datos:

- Instalación de PostgreSQL.
- Creación de la base de datos.
- Configuración de variables de entorno.
- Primera conexión Python → PostgreSQL.

Capítulo 2

Hito 2: Instalación y Configuración de PostgreSQL

2.1. Objetivo

El objetivo de este hito es preparar el motor de base de datos que servirá como núcleo del Data Warehouse macrofinanciero.

Al finalizar este hito el estudiante será capaz de:

- Instalar PostgreSQL.
- Crear una base de datos para el proyecto.
- Conectarse desde VS Code.
- Configurar variables de entorno.
- Conectarse desde Python.
- Ejecutar consultas SQL básicas.

2.2. Resultado esperado

Al finalizar este hito deberá existir:

- Una instancia funcional de PostgreSQL.
- Una base de datos llamada **macrofinanzas**.
- Un usuario con permisos adecuados.

CAPÍTULO 2. HITO 2: INSTALACIÓN Y CONFIGURACIÓN DE POSTGRESQL14

- Un archivo `.env` para gestionar credenciales.
- Un script Python capaz de conectarse a la base de datos.

2.3. Paso 1: Verificar instalación de PostgreSQL

Abrir PowerShell y ejecutar:

```
psql --version
```

Salida esperada:

```
psql (PostgreSQL) 17.x
```

Si el comando no funciona:

- Verificar que PostgreSQL está instalado.
- Verificar que el directorio `bin` está en el `PATH`.

2.4. Paso 2: Iniciar PostgreSQL

Verificar que el servicio está activo.

En Windows:

1. Abrir Servicios.
2. Buscar PostgreSQL.
3. Verificar estado Running".

Alternativamente:

```
net start postgresql-x64-17
```

2.5. Paso 3: Conectarse al servidor

Abrir terminal:

```
psql -U postgres
```

Introducir la contraseña configurada durante la instalación.

Debería aparecer:

```
postgres=#
```

2.6. Paso 4: Crear la base de datos

Ejecutar:

```
CREATE DATABASE macrofinanzas;
```

Verificar:

```
\l
```

La nueva base deberá aparecer en la lista.

Salir:

```
\q
```

2.7. Paso 5: Conectarse a la nueva base

```
psql -U postgres -d macrofinanzas
```

Verificar:

```
SELECT current_database();
```

Resultado esperado:

```
macrofinanzas
```

2.8. Paso 6: Crear archivo .env

En la raíz del proyecto crear:

```
.env
```

Contenido:

```
DB_HOST=localhost  
DB_PORT=5432  
DB_NAME=macrofinanzas  
DB_USER=postgres  
DB_PASSWORD=tu_password
```

Importante:

El archivo `.env` nunca debe enviarse a GitHub.

2.9. Paso 7: Actualizar .env.example

Crear una versión pública:

`.env.example`

Contenido:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=macrofinanzas
DB_USER=postgres
DB_PASSWORD=your_password
```

Este archivo sí puede subirse al repositorio.

2.10. Paso 8: Crear módulo de conexión

Crear:

`etl/db.py`

```
from sqlalchemy import create_engine
from dotenv import load_dotenv
import os

load_dotenv()

HOST = os.getenv("DB_HOST")
PORT = os.getenv("DB_PORT")
DB = os.getenv("DB_NAME")
USER = os.getenv("DB_USER")
PASSWORD = os.getenv("DB_PASSWORD")

connection_string = (
    f"postgresql+psycopg2://{USER}:{PASSWORD}@{HOST}:{PORT}/{DB}"
)

engine = create_engine(connection_string)

print("Conexion_creada_correctamente.")
```

2.11. Paso 9: Probar la conexión

Ejecutar:

```
python etl/db.py
```

Salida esperada:

```
Conexión creada correctamente.
```

2.12. Paso 10: Realizar una consulta desde Python

Modificar temporalmente:

```
from sqlalchemy import text

with engine.connect() as conn:
    resultado = conn.execute(
        text("SELECT version();")
    )

    '''
    for fila in resultado:
        print(fila)
    '''
```

Ejecutar nuevamente:

```
python etl/db.py
```

Resultado esperado:

```
PostgreSQL 17.x ...
```

2.13. Paso 11: Instalar extensiones de VS Code

Instalar:

- PostgreSQL
- SQLTools
- SQLTools PostgreSQL Driver

Estas extensiones facilitarán la ejecución de consultas y exploración de objetos.

2.14. Paso 12: Crear primer script SQL

Crear:

`sql/test_connection.sql`

Contenido:

SELECT current_database();

SELECT current_user;

SELECT version();

Ejecutarlo desde VS Code o psql.

2.15. Checklist de validación

Antes de continuar verificar:

- ☐ PostgreSQL instalado.
- ☐ Servicio iniciado.
- ☐ Base de datos creada.
- ☐ Archivo .env configurado.
- ☐ Python conectado a PostgreSQL.
- ☐ Consulta SQL ejecutada correctamente.

2.16. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Base de datos **macrofinanzas**.
- Módulo de conexión Python.
- Variables de entorno configuradas.
- VS Code conectado a PostgreSQL.
- Primer script SQL ejecutado exitosamente.

2.17. Próximo hito

En el Hito 3 construiremos la primera capa de datos:

- Diseño de las fuentes.
- Creación de datos simulados.
- Organización de carpetas **raw** y **processed**.
- Primera ingesta hacia PostgreSQL.

Capítulo 3

Hito 2: Configuración de DuckDB como Base de Datos Analítica Local

3.1. Objetivo

El objetivo de este hito es preparar la base de datos analítica que servirá como núcleo del proyecto.

A diferencia de PostgreSQL, DuckDB no requiere instalación como servicio ni privilegios de administrador. La base de datos se almacena como un archivo local dentro del proyecto, lo cual la convierte en una excelente opción para trabajar en laptops corporativas con restricciones de instalación.

Al finalizar este hito el estudiante será capaz de:

- Instalar DuckDB desde Python.
- Crear una base de datos local en formato `.duckdb`.
- Conectarse a DuckDB desde Python.
- Ejecutar consultas SQL básicas.
- Crear un módulo reutilizable de conexión.
- Verificar que el proyecto puede operar sin PostgreSQL.

3.2. Resultado esperado

Al finalizar este hito deberá existir una base de datos local:

```
data/warehouse/macrofinanzas.duckdb
```

y un archivo de conexión:

```
etl/db.py
```

capaz de abrir la base de datos desde Python.

3.3. ¿Por qué DuckDB?

DuckDB es una base de datos analítica embebida.

Esto significa que:

- No requiere servidor.
- No requiere instalación con privilegios de administrador.
- No necesita usuarios ni contraseñas.
- Puede vivir como un archivo dentro del proyecto.
- Se integra directamente con Python, Pandas y Streamlit.
- Es muy eficiente para consultas analíticas.

En este proyecto utilizaremos DuckDB como motor local para construir el Data Warehouse macrofinanciero.

3.4. Arquitectura del hito

La nueva arquitectura será:

```
CSV
|
V
Python
|
V
```

CAPÍTULO 3. HITO 2: CONFIGURACIÓN DE DUCKDB COMO BASE DE DATOS ANALÍTICA LO

```
DuckDB
|
V
Streamlit
```

La base quedará almacenada como archivo:

```
macrofinanzas.duckdb
```

3.5. Paso 1: Actualizar la estructura del proyecto

Dentro del proyecto crear la carpeta:

```
data/warehouse
```

En PowerShell:

```
mkdir data\warehouse
```

La estructura deberá quedar así:

```
dwh_macrofinanciero_rd/

+-- data/
|   +-- raw/
|   +-- processed/
|   -- warehouse/

+-- sql/
+-- etl/
+-- app/
+-- docs/
+-- tests/
```

3.6. Paso 2: Instalar DuckDB

Con el entorno virtual activo:

```
pip install duckdb
```

También instalaremos paquetes que utilizaremos en el proyecto:

```
pip install pandas
pip install python-dotenv
```

Guardar dependencias:

```
pip freeze > requirements.txt
```

3.7. Paso 3: Verificar instalación

Ejecutar:

```
python
```

Dentro de Python:

```
import duckdb
```

```
print(duckdb.**version**)
```

Salir:

```
exit()
```

Si se imprime la versión, DuckDB está correctamente instalado.

3.8. Paso 4: Crear archivo de configuración

Aunque DuckDB no requiere usuario ni contraseña, utilizaremos un archivo `.env` para mantener centralizada la ruta de la base.

Crear:

```
.env
```

Contenido:

```
DUCKDB_PATH=data/warehouse/macrofinanzas.duckdb
```

Importante: el archivo `.env` no debe subirse a GitHub.

3.9. Paso 5: Actualizar `.env.example`

Crear o actualizar:

```
.env.example
```


CAPÍTULO 3. HITO 2: CONFIGURACIÓN DE DUCKDB COMO BASE DE DATOS ANALÍTICA LO

Contenido:

DUCKDB_PATH=data/warehouse/macrofinanzas.duckdb

Este archivo sí puede subirse al repositorio.

3.10. Paso 6: Crear módulo de conexión

Crear:

etl/db.py

Contenido:

```
import os
import duckdb

from dotenv import load_dotenv

load_dotenv()

DB_PATH = os.getenv(
    "DUCKDB_PATH",
    "data/warehouse/macrofinanzas.duckdb"
)

def get_connection():
    conn = duckdb.connect(DB_PATH)
    return conn

if __name__ == "__main__":
    conn = get_connection()

    """
    result = conn.execute(
        "SELECT_ 'DuckDB_conectado_correctamente '_AS_mensaje;"
    ).fetchdf()

    print(result)

    conn.close()
    """
```

3.11. Paso 7: Ejecutar prueba de conexión

Desde la terminal:

```
python etl/db.py
```

Salida esperada:

```
mensaje
0 DuckDB conectado correctamente
```

Al ejecutar este script, DuckDB creará automáticamente el archivo:

```
data/warehouse/macrofinanzas.duckdb
```

3.12. Paso 8: Verificar archivo de base de datos

Confirmar que existe:

```
data/warehouse/macrofinanzas.duckdb
```

En PowerShell:

```
dir data\warehouse
```

Deberá aparecer el archivo `macrofinanzas.duckdb`.

3.13. Paso 9: Crear primer script SQL

Crear:

```
sql/test_connection.sql
```

Contenido:

```
SELECT
'DuckDB' AS motor,
current_database() AS base_actual;
```

3.14. Paso 10: Ejecutar SQL desde Python

Crear:

etl/run_sql_file.py

Contenido:

```
from db import get_connection

def run_sql_file(path):
    conn = get_connection()

    """
    with open(path, "r", encoding="utf-8") as file:
        sql = file.read()

    result = conn.execute(sql).fetchdf()

    print(result)

    conn.close()
    """

if __name__ == "__main__":
    run_sql_file(
        "sql/test_connection.sql"
    )
```

Ejecutar:

```
python etl/run_sql_file.py
```

Salida esperada:

```
motor          base_actual
0  DuckDB      macrofinanzas
```

3.15. Paso 11: Instalar extensión recomendada en VS Code

Buscar e instalar en VS Code:

CAPÍTULO 3. HITO 2: CONFIGURACIÓN DE DUCKDB COMO BASE DE DATOS ANALÍTICA LOCAL

- DuckDB SQL Tools
- Python

Esto permitirá explorar archivos DuckDB y ejecutar consultas desde el editor.

3.16. Paso 12: Crear consulta de prueba

Crear:

`sql/check_duckdb.sql`

Contenido:

```
SELECT  
1 AS prueba ,  
'ok' AS estado ;
```

Modificar temporalmente `etl/run_sql_file.py` para ejecutar este archivo o reutilizar la función:

```
run_sql_file(  
    "sql/check_duckdb.sql"  
)
```

3.17. Diferencias importantes frente a PostgreSQL

Al usar DuckDB, algunas cosas cambian:

- No necesitamos servidor.
- No necesitamos usuarios.
- No usamos contraseñas.
- No dependemos del puerto 5432.
- La base vive en un archivo.
- El proyecto es más portable.

Sin embargo, mantendremos la lógica conceptual del Data Warehouse:

CAPÍTULO 3. HITO 2: CONFIGURACIÓN DE DUCKDB COMO BASE DE DATOS ANALÍTICA LO

staging -> core -> mart

DuckDB soporta schemas, por lo que seguiremos trabajando con:

- staging
- core
- mart

3.18. Buenas prácticas

Durante este proyecto seguiremos estas reglas:

1. La base DuckDB se guardará en **data/warehouse**.
2. El archivo **.env** guardará la ruta local.
3. El archivo **.env.example** documentará la configuración.
4. El módulo **etl/db.py** será el único responsable de abrir conexiones.
5. Cada script cerrará la conexión al finalizar.

3.19. Checklist de validación

Antes de continuar verificar:

- ☐ Carpeta **data/warehouse** creada.
- ☐ DuckDB instalado.
- ☐ Archivo **.env** creado.
- ☐ Archivo **.env.example** actualizado.
- ☐ Módulo **etl/db.py** creado.
- ☐ Script **etl/db.py** ejecutado correctamente.
- ☐ Archivo **macrofinanzas.duckdb** generado.
- ☐ Script SQL de prueba ejecutado.

3.20. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Una base DuckDB local.
- Un módulo de conexión reutilizable.
- Una carpeta `data/warehouse`.
- Un archivo `.env.example`.
- Un mecanismo básico para ejecutar SQL desde Python.

3.21. Próximo hito

En el Hito 3 trabajaremos con las primeras fuentes de datos del proyecto. Construiremos:

- Archivos CSV iniciales.
- Lectura desde Python.
- Validaciones básicas.
- Organización de datos crudos y procesados.

Capítulo 4

Hito 3: Diseño de las Fuentes de Datos e Ingesta Inicial

4.1. Objetivo

El objetivo de este hito es construir la primera capa del flujo de datos. Al finalizar este hito el estudiante será capaz de:

- Comprender las fuentes de información del proyecto.
- Diseñar datasets alineados con las necesidades de negocio.
- Organizar correctamente las carpetas de datos.
- Crear archivos CSV simulados.
- Leer datos desde Python.
- Realizar validaciones básicas antes de cargar a PostgreSQL.

4.2. ¿Por qué comenzamos con archivos CSV?

En proyectos reales los datos suelen provenir de:

- APIs.
- Bases de datos.
- Excel.

- CSV.
- Servicios web.
- Plataformas regulatorias.

Sin embargo, durante las primeras etapas del proyecto es recomendable trabajar con archivos CSV.

Esto permite:

- Diseñar el modelo de datos.
- Construir el ETL.
- Validar la arquitectura.
- Evitar complejidad innecesaria.

Una vez validado el proceso, sustituiremos los CSV por fuentes reales.

4.3. Arquitectura de la ingesta

El flujo de trabajo será:

```
Fuente CSV
|
V
data/raw
|
V
Python
|
V
Validación
|
V
data/processed
|
V
PostgreSQL
```


4.4. Fuentes utilizadas

El proyecto utilizará dos fuentes principales.

4.4.1. Fuente 1: Tipo de Cambio

Representa información similar a la publicada por el Banco Central.

Grano:

Una observación por fecha y moneda.

Variables:

- fecha
- moneda
- tasa_compra
- tasa_venta

Ejemplo:

```
fecha,moneda,tasa_compra,tasa_venta
2024-01-01,USD,58.50,58.80
2024-01-01,EUR,63.10,63.60
```

4.4.2. Fuente 2: Balances Bancarios

Representa información similar a la publicada por la Superintendencia de Bancos.

Grano:

Una observación por mes y entidad.

Variables:

- periodo
- entidad_cod
- entidad_nombre
- tipo_entidad

- activos_totales
- cartera_creditos
- depositos
- patrimonio
- resultado_neto
- depositos_me
- cartera_me

Ejemplo:

```
periodo,entidad_cod,activos_totales
202401,B001,120000000
```

4.5. Paso 1: Crear estructura de almacenamiento

Verificar que existe:

```
data/
```

```
+-- raw/
```

```
+-- processed/
```

`raw` contendrá los datos originales.

`processed` contendrá los datos validados.

4.6. Paso 2: Crear archivo tipo_cambio.csv

Crear:

```
data/raw/tipo_cambio.csv
```

Contenido mínimo:

```
fecha,moneda,tasa_compra,tasa_venta
2024-01-01,USD,58.50,58.80
2024-01-02,USD,58.55,58.85
2024-01-03,USD,58.60,58.90
2024-01-01,EUR,63.10,63.60
2024-01-02,EUR,63.20,63.70
```

4.7. Paso 3: Crear archivo balances.csv

Crear:

data/raw/balances.csv

Contenido mínimo:

```
periodo,entidad_cod,entidad_nombre,  
tipo_entidad,activos_totales,  
cartera_creditos,depositos,  
patrimonio,resultado_netto,  
depositos_me,cartera_me
```

```
202401,B001,Banco Alfa,  
Banco Multiple,120000000,  
85000000,95000000,  
15000000,1200000,  
25000000,18000000
```

4.8. Paso 4: Crear script de exploración

Crear:

etl/explore_sources.py

```
import pandas as pd  
  
fx = pd.read_csv(  
    "data/raw/tipo_cambio.csv"  
)  
  
balances = pd.read_csv(  
    "data/raw/balances.csv"  
)  
  
print("\nTipo_de_Cambio")  
print(fx.head())  
  
print("\nBalances")  
print(balances.head())
```

4.9. Paso 5: Ejecutar exploración

Ejecutar:

```
python etl/explore_sources.py
```

Resultado esperado:

- Visualización de las primeras filas.
- Confirmación de lectura correcta.

4.10. Paso 6: Verificar estructura

Agregar:

```
print (fx . info ())
```

```
print (balances . info ())
```

Analizar:

- Número de filas.
- Número de columnas.
- Tipos de datos.
- Valores faltantes.

4.11. Paso 7: Validaciones básicas

Agregar:

```
print (
fx . isnull ().sum()
)
```

```
print (
balances . isnull ().sum()
)
```

Verificar:

- Valores nulos.

- Fechas inválidas.
- Tasas negativas.

4.12. Paso 8: Crear capa processed

Convertir tipos de datos:

```
fx["fecha"] = pd.to_datetime(
fx["fecha"]
)

fx["tasa_compra"] = (
fx["tasa_compra"]
. astype(float)
)

fx["tasa_venta"] = (
fx["tasa_venta"]
. astype(float)
)
```

4.13. Paso 9: Guardar datos procesados

```
fx.to_csv(
"data/processed/tipo_cambio.csv",
index=False
)

balances.to_csv(
"data/processed/balances.csv",
index=False
)
```

4.14. Paso 10: Documentar las fuentes

Crear:

```
docs/data_dictionary.md
```

Documentar:

- Nombre de variable.
- Descripción.
- Tipo de dato.
- Fuente.
- Frecuencia.

4.15. Entregables

Al finalizar este hito deberá existir:

- Archivos CSV en raw.
- Archivos validados en processed.
- Script de exploración.
- Diccionario de datos.
- Validaciones básicas implementadas.

4.16. Checklist de validación

- ☐ Existe carpeta raw.
- ☐ Existe carpeta processed.
- ☐ tipo_cambio.csv creado.
- ☐ balances.csv creado.
- ☐ Python puede leer ambos archivos.
- ☐ Validaciones ejecutadas.
- ☐ Archivos procesados generados.

4.17. Próximo hito

En el Hito 4 diseñaremos el modelo dimensional.
Construiremos:

- Dimensión Tiempo.
- Dimensión Moneda.
- Dimensión Entidad Financiera.
- Hecho Tipo de Cambio.
- Hecho Balance Bancario.
- Primer esquema estrella.

Capítulo 5

Hito 4: Diseño del Modelo Dimensional

5.1. Objetivo

El objetivo de este hito es diseñar el modelo dimensional del Data Warehouse macrofinanciero.

Al finalizar este hito el estudiante será capaz de:

- Identificar el grano de cada tabla de hechos.
- Diferenciar dimensiones y hechos.
- Diseñar un esquema estrella.
- Definir claves de negocio y claves subrogadas.
- Documentar el modelo antes de implementarlo en PostgreSQL.

5.2. Resultado esperado

Al finalizar este hito tendremos definido el modelo lógico del Data Warehouse.

El modelo estará compuesto por:

- `dim_tiempo`
- `dim_moneda`
- `dim_entidad_financiera`

- `fact_tipo_cambio`
- `fact_balance_mensual`

5.3. Paso 1: Entender qué es un modelo dimensional

Un modelo dimensional organiza los datos para facilitar el análisis. Está compuesto por dos tipos principales de tablas:

- **Dimensiones:** describen el contexto del análisis.
- **Hechos:** contienen medidas numéricas que queremos analizar.

Ejemplo:

- La fecha es una dimensión.
- La entidad financiera es una dimensión.
- La moneda es una dimensión.
- La tasa de cambio es una medida.
- Los activos bancarios son una medida.

5.4. Paso 2: Identificar los procesos de negocio

En este proyecto tendremos dos procesos de negocio principales:

1. Seguimiento del mercado cambiario.
2. Seguimiento de balances bancarios mensuales.

Cada proceso generará una tabla de hechos.

5.5. Paso 3: Definir el grano

El grano indica qué representa una fila de una tabla de hechos. Esta decisión es fundamental.

5.5.1. Grano de fact_tipo_cambio

Una fila por fecha y moneda.

Ejemplo:

```
2024-01-01 | USD
2024-01-01 | EUR
2024-01-02 | USD
```

5.5.2. Grano de fact_balance_mensual

Una fila por mes y entidad financiera.

Ejemplo:

```
2024-01-31 | B001
2024-01-31 | B002
2024-02-29 | B001
```

5.6. Paso 4: Definir dimensiones**5.6.1. Dimensión tiempo**

La dimensión tiempo será compartida por ambos hechos.
Esto la convierte en una dimensión conformada.

```
dim\_tiempo
```

```
sk\_tiempo
fecha
anio
mes
anio\_mes
nombre\_mes
trimestre
es\_fin\_de_mes
```

5.6.2. Dimensión moneda

Esta dimensión describe las monedas utilizadas en el mercado cambiario.

```
dim\_moneda

sk\_moneda
moneda\_cod
moneda\_nombre
```

5.6.3. Dimensión entidad financiera

Esta dimensión describe las entidades financieras.

```
dim\_entidad\_financiera

sk\_entidad
entidad\_cod
entidad\_nombre
tipo\_entidad
vigente\_desde
vigente\_hasta
es\_actual
```

La dimensión entidad financiera se diseñará para soportar cambios históricos mediante SCD Tipo 2.

5.7. Paso 5: Definir tablas de hechos**5.7.1. Hecho tipo de cambio**

```
fact\_tipo\_cambio

sk\_tiempo
sk\_moneda
tasa_compra
tasa_venta
tasa_promedio
spread
```

Medidas:

- tasa_compra
- tasa_venta
- tasa_promedio
- spread

5.7.2. Hecho balance mensual

fact_balance_mensual

sk_tiempo
 sk_entidad
 activos_totales
 cartera_creditos
 depositos
 patrimonio
 resultado_neto
 depositos_me
 cartera_me

Medidas:

- activos_totales
- cartera_creditos
- depositos
- patrimonio
- resultado_netto
- depositos_me
- cartera_me

5.8. Paso 6: Definir claves

5.8.1. Clave de negocio

La clave de negocio proviene de la fuente original.
 Ejemplos:

- moneda_cod: USD, EUR.
- entidad_cod: B001, B002.
- fecha 2024-01-31.

5.8.2. Clave subrogada

La clave subrogada es creada por el Data Warehouse.
Ejemplos:

- sk_tiempo
- sk_moneda
- sk_entidad

Las tablas de hechos no deben depender directamente de las claves de negocio.

Deben conectarse mediante claves subrogadas.

5.9. Paso 7: Diseñar el esquema estrella

El esquema estrella del proyecto será:

```

dim_tiempo
|
|
dim_moneda ---- fact_tipo_cambio

      dim_tiempo
      |
      |

dim_entidad_financiera ---- fact_balance_mensual

```

La dimensión `dim_tiempo` conecta ambos hechos y permite análisis integrados.

5.10. Paso 8: Crear documento del modelo

Crear el archivo:

docs/modelo_dimensional.md

Contenido sugerido:

```
# Modelo Dimensional
```

```
## Procesos de negocio
```

1. Mercado cambiario
2. Balances bancarios mensuales

```
## Tablas de hechos
```

```
### fact_tipo_cambio
```

Grano:

Una fila por fecha y moneda.

Medidas:

- * tasa_compra
- * tasa_venta
- * tasa_promedio
- * spread

```
### fact_balance_mensual
```

Grano:

Una fila por mes y entidad financiera.

Medidas:

- * activos_totales
- * cartera_creditos
- * depositos
- * patrimonio

```

* resultado_netto
* depositos_me
* cartera_me

## Dimensiones

* dim_tiempo
* dim_moneda
* dim_entidad_financiera

```

5.11. Paso 9: Crear tabla de definición del modelo

Agregar al documento una tabla como la siguiente:

Tabla	Tipo	Grano
dim_tiempo	Dimensión	Una fila por día
dim_moneda	Dimensión	Una fila por moneda
dim_entidad_financiera	Dimensión	Una fila por versión de entidad
fact_tipo_cambio	Hecho	Una fila por fecha y moneda
fact_balance_mensual	Hecho	Una fila por mes y entidad

5.12. Paso 10: Definir indicadores derivados

A partir de este modelo construiremos indicadores como:

- Tipo de cambio promedio.
- Spread cambiario.
- ROA.
- ROE.
- Dolarización de depósitos.
- Dolarización de cartera.
- Concentración bancaria.

Fórmulas principales:

```

tasa_promedio = (tasa_compra + tasa_venta) / 2

spread = tasa_venta - tasa_compra

ROA = resultado_netto / activos_totales

ROE = resultado_netto / patrimonio

dolarizacion_depositos = depositos_me / depositos

dolarizacion_cartera = cartera_me / cartera_creditos

```

5.13. Paso 11: Validar el diseño

Antes de crear las tablas en PostgreSQL, responder:

1. ¿Cada tabla de hechos tiene un grano claro?
2. ¿Cada medida pertenece al grano definido?
3. ¿Las dimensiones describen correctamente los hechos?
4. ¿La dimensión tiempo puede usarse en ambos hechos?
5. ¿La entidad financiera requiere historia?

5.14. Checklist de validación

Antes de continuar verificar:

- ☐ Procesos de negocio identificados.
- ☐ Grano de cada hecho definido.
- ☐ Dimensiones definidas.
- ☐ Hechos definidos.
- ☐ Claves de negocio identificadas.
- ☐ Claves subrogadas definidas.
- ☐ Documento `modelo_dimensional.md` creado.

5.15. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Diseño lógico del Data Warehouse.
- Documento del modelo dimensional.
- Definición de grano.
- Lista de dimensiones.
- Lista de hechos.
- Lista de indicadores derivados.

5.16. Próximo hito

En el Hito 5 implementaremos la primera capa física del Data Warehouse:

- Creación de schemas.
- Creación de tablas staging.
- Preparación de la capa bronze.
- Primer script SQL del modelo.

Capítulo 6

Hito 5: Construcción de la Capa Staging (Bronze)

6.1. Objetivo

El objetivo de este hito es construir la primera capa física del Data Warehouse.

La capa staging será el punto de entrada de todos los datos al sistema. Al finalizar este hito el estudiante será capaz de:

- Crear schemas en PostgreSQL.
- Diseñar tablas de aterrizaje.
- Comprender la función de la capa Bronze.
- Implementar una estrategia de carga desacoplada de las transformaciones.
- Preparar el sistema para recibir datos desde Python.

6.2. Resultado esperado

Al finalizar este hito deberá existir la siguiente estructura lógica:

PostgreSQL

```
+-- staging  
|
```

```
+-- core
|
+-- mart
```

Y dentro del schema staging:

```
staging.tipo_cambio_raw

staging.balance_bancario_raw
```

6.3. ¿Qué es la capa Staging?

La capa staging es el área donde aterrizan los datos provenientes de las fuentes externas.

Su objetivo es almacenar los datos en su forma más cercana al origen.

No se realizan:

- Cálculos.
- Transformaciones complejas.
- Generación de indicadores.
- Limpieza profunda.

La regla general es:

Cargar primero. Transformar después.

6.4. Arquitectura del Data Warehouse

La arquitectura que construiremos será:

```
Fuentes
|
V
Staging (Bronze)
|
V
Core (Silver)
|
V
Mart (Gold)
```

6.5. Paso 1: Crear carpeta para scripts SQL

Verificar:

```
sql/
```

Crear:

```
sql/01_create_schemas.sql
```

6.6. Paso 2: Crear los schemas

Agregar:

```
CREATE SCHEMA IF NOT EXISTS staging;
```

```
CREATE SCHEMA IF NOT EXISTS core;
```

```
CREATE SCHEMA IF NOT EXISTS mart;
```

Guardar.

6.7. Paso 3: Ejecutar el script

Desde psql:

```
\i sql/01_create_schemas.sql
```

Verificar:

```
SELECT schema_name
FROM information_schema.schemata
WHERE schema_name IN
(
  'staging',
  'core',
  'mart'
);
```

Resultado esperado:

```
staging
core
mart
```

6.8. Paso 4: Crear script de tablas staging

Crear:

sql/02_staging_ddl.sql

6.9. Paso 5: Diseñar tabla tipo_cambio_raw

Agregar:

```
CREATE TABLE IF NOT EXISTS
staging.tipo_cambio_raw
(
  fecha TEXT,

  '''
moneda TEXT,

tasa_compra TEXT,

tasa_venta TEXT,

archivo_origen TEXT,

cargado_en TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
'''

);
```

6.10. ¿Por qué usamos TEXT?

Esta es una práctica común en Data Engineering.

Ventajas:

- La carga nunca falla por formato.
- Se preserva el dato original.
- La validación ocurre posteriormente.
- Se facilita el diagnóstico de errores.

6.11. Paso 6: Diseñar tabla balance_bancario_raw

Agregar:

```
CREATE TABLE IF NOT EXISTS
staging.balance_bancario_raw
(
periodo TEXT,

'''
entidad_cod TEXT,

entidad_nombre TEXT,

tipo_entidad TEXT,

activos_totales TEXT,

cartera_creditos TEXT,

depositos TEXT,

patrimonio TEXT,

resultado_netto TEXT,

depositos_me TEXT,

cartera_me TEXT,

archivo_origen TEXT,

cargado_en TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
'''

);
```

6.12. Paso 7: Ejecutar el DDL

Ejecutar:

```
\i sql/02_staging_ddl.sql
```

6.13. Paso 8: Verificar tablas creadas

Consultar:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema='staging';
```

Resultado esperado:

```
tipo_cambio_raw
```

```
balance_bancario_raw
```

6.14. Paso 9: Inspeccionar estructura

Consultar:

```
SELECT
column_name,
data_type
FROM information_schema.columns
WHERE table_schema='staging'
AND table_name='tipo_cambio_raw';
```

Verificar que todas las columnas fueron creadas correctamente.

6.15. Paso 10: Crear script de prueba

Crear:

```
sql/test_staging.sql
```

Contenido:

```
SELECT *  
FROM staging.tipo_cambio_raw;
```

```
SELECT *  
FROM staging.balance_bancario_raw;
```

Actualmente ambas tablas deben estar vacías.

6.16. Paso 11: Documentar la capa staging

Crear:

docs/staging_design.md

Documentar:

- Nombre de tabla.
- Fuente original.
- Frecuencia.
- Número esperado de columnas.
- Responsable de carga.

Ejemplo:

Tabla:
staging.tipo_cambio_raw

Fuente:
Banco Central

Frecuencia:
Diaria

Objetivo:
Almacenar datos crudos del mercado cambiario.

6.17. Paso 12: Crear consulta de monitoreo

Crear:

sql/monitor_staging.sql

Contenido:

```
SELECT  
COUNT(*) AS total_filas  
FROM staging.tipo_cambio_raw;
```

```
SELECT  
COUNT(*) AS total_filas  
FROM staging.balance_bancario_raw;
```

Esta consulta se utilizará para monitorear futuras cargas.

6.18. Buenas prácticas

Durante este proyecto seguiremos las siguientes reglas:

1. Nunca modificar datos manualmente en staging.
2. Mantener los datos lo más cercanos posible a la fuente.
3. No realizar cálculos en staging.
4. Todas las transformaciones ocurren en la capa core.
5. Conservar metadatos de carga.

6.19. Checklist de validación

Antes de continuar verificar:

- ☐ Schema staging creado.
- ☐ Schema core creado.
- ☐ Schema mart creado.
- ☐ Tabla `tipo_cambio_raw` creada.

- ☐ Tabla `balance_bancario_raw` creada.
- ☐ Scripts SQL almacenados en la carpeta `sql`.
- ☐ Consultas de validación ejecutadas.

6.20. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Arquitectura física inicial del Data Warehouse.
- Schemas creados.
- Tablas staging creadas.
- Scripts SQL documentados.
- Capa Bronze lista para recibir datos.

6.21. Próximo hito

En el Hito 6 construiremos el primer proceso ETL.
Aprenderemos a:

- Leer archivos CSV desde Python.
- Conectarnos a PostgreSQL.
- Cargar información a staging.
- Automatizar la ingesta.
- Validar que los datos fueron cargados correctamente.

Capítulo 7

Hito 5: Construcción de la Capa Staging en DuckDB

7.1. Objetivo

El objetivo de este hito es construir la primera capa física del Data Warehouse utilizando DuckDB.

La capa **staging** será el punto de entrada de todos los datos al sistema. En esta capa almacenaremos los datos provenientes de archivos CSV en una forma cercana a la fuente original.

Al finalizar este hito el estudiante será capaz de:

- Crear schemas en DuckDB.
- Crear tablas de aterrizaje para datos crudos.
- Ejecutar scripts SQL desde Python.
- Preparar la base local para recibir datos desde archivos CSV.
- Mantener la arquitectura **staging ->core ->mart**.

7.2. Resultado esperado

Al finalizar este hito deberán existir tres schemas:

```
staging
core
mart
```

y dos tablas en la capa `staging`:

```
staging.tipo_cambio_raw
```

```
staging.balance_bancario_raw
```

7.3. Paso 1: Crear script de schemas

Crear el archivo:

```
sql/01_create_schemas.sql
```

Contenido:

```
CREATE SCHEMA IF NOT EXISTS staging;
```

```
CREATE SCHEMA IF NOT EXISTS core;
```

```
CREATE SCHEMA IF NOT EXISTS mart;
```

7.4. Paso 2: Crear script para ejecutar SQL

Crear o verificar el archivo:

```
etl/run_sql_file.py
```

Contenido:

```
from db import get_connection
```

```
def run_sql_file(path):
```

```
    '''
```

```
    conn = get_connection()
```

```
    with open(path, "r", encoding="utf-8") as file:
        sql = file.read()
```

```
    conn.execute(sql)
```

```
    conn.close()
```

```
'''  
  
if __name__ == "__main__":  
  
    '''  
    run_sql_file(  
        "sql/01_create_schemas.sql"  
    )  
  
    print("Script_SQL_ejecutado_correctamente.")  
'''
```

7.5. Paso 3: Ejecutar creación de schemas

Desde la terminal:

```
python etl/run_sql_file.py
```

Salida esperada:

```
Script SQL ejecutado correctamente.
```

7.6. Paso 4: Verificar schemas

Crear el archivo:

```
sql/check_schemas.sql
```

Contenido:

```
SELECT schema_name  
FROM information_schema.schemata  
WHERE schema_name IN  
(  
    'staging',  
    'core',  
    'mart'  
)  
ORDER BY schema_name;
```

Modificar temporalmente `etl/run_sql_file.py` para ejecutar :

CAPÍTULO 7. HITO 5: CONSTRUCCIÓN DE LA CAPA STAGING EN DUCKDB61

```
run_sql_file(  
  "sql/check_schemas.sql"  
)
```

Resultado esperado:

```
core  
mart  
staging
```

7.7. Paso 5: Crear script de tablas staging

Crear:

```
sql/02_staging_ddl.sql
```

7.8. Paso 6: Crear tabla tipo *cambio_raw*

Agregar:

```
CREATE TABLE IF NOT EXISTS staging.tipo_cambio_raw  
(  
  fecha TEXT,  
  
  '''  
  moneda TEXT,  
  
  tasa_compra TEXT,  
  
  tasa_venta TEXT,  
  
  archivo_origen TEXT,  
  
  cargado_en TIMESTAMP  
  ''')  
);
```

7.9. Paso 7: Crear tabla `balance_bancario_raw`

Agregar:

```
CREATE TABLE IF NOT EXISTS staging.balance_bancario_raw
(
  periodo TEXT,

  '''
  entidad_cod TEXT,

  entidad_nombre TEXT,

  tipo_entidad TEXT,

  activos_totales TEXT,

  cartera_creditos TEXT,

  depositos TEXT,

  patrimonio TEXT,

  resultado_netto TEXT,

  depositos_me TEXT,

  cartera_me TEXT,

  archivo_origen TEXT,

  cargado_en TIMESTAMP
  '''
);
```

7.10. Nota sobre tipos de datos

En staging usaremos columnas tipo TEXT.

La idea es cargar primero los datos crudos y validar después.

CAPÍTULO 7. HITO 5: CONSTRUCCIÓN DE LA CAPA STAGING EN DUCKDB63

Esta decisión permite:

- Evitar fallos tempranos por formatos incorrectos.
- Conservar el dato original.
- Separar carga de transformación.
- Diagnosticar errores con mayor facilidad.

7.11. Paso 8: Ejecutar DDL de staging

Modificar temporalmente:

etl/run_sql_file.py

para ejecutar:

```
run_sql_file(  
    "sql/02_staging_ddl.sql"  
)
```

Luego ejecutar:

```
python etl/run_sql_file.py
```

7.12. Paso 9: Verificar tablas creadas

Crear:

sql/check_staging_tables.sql

Contenido:

```
SELECT table_schema,  
       table_name  
FROM information_schema.tables  
WHERE table_schema = 'staging'  
ORDER BY table_name;
```

Resultado esperado:

```
staging    balance_bancario_raw  
staging    tipo_cambio_raw
```


7.13. Paso 10: Crear script general para inicializar la base

Crear:

etl/init_database.py

Contenido:

```
from db import get_connection

def run_sql(path, conn):

    '''
    with open(path, "r", encoding="utf-8") as file:
        sql = file.read()

    conn.execute(sql)
    '''

def main():

    '''
    conn = get_connection()

    run_sql(
        "sql/01_create_schemas.sql",
        conn
    )

    run_sql(
        "sql/02_staging_ddl.sql",
        conn
    )

    conn.close()

    print("Base DuckDB inicializada correctamente.")
    '''

if __name__ == "__main__":
```

```
main()
```

7.14. Paso 11: Ejecutar inicialización completa

Desde la terminal:

```
python etl/init_database.py
```

Salida esperada:

Base DuckDB inicializada correctamente.

7.15. Paso 12: Crear consulta de monitoreo

Crear:

```
sql/monitor_staging.sql
```

Contenido:

```
SELECT
'tipo_cambio_raw' AS tabla,
COUNT(*) AS total_filas
FROM staging.tipo_cambio_raw
```

```
UNION ALL
```

```
SELECT
'balance_bancario_raw' AS tabla,
COUNT(*) AS total_filas
FROM staging.balance_bancario_raw;
```

Inicialmente ambas tablas deberán tener cero filas.

7.16. Buenas prácticas

Durante este hito seguiremos estas reglas:

1. La base DuckDB vive en data/warehouse.
2. Los schemas separan las capas del Data Warehouse.

CAPÍTULO 7. HITO 5: CONSTRUCCIÓN DE LA CAPA STAGING EN DUCKDB66

3. La capa staging recibe datos crudos.
4. Las transformaciones ocurrirán en core.
5. Las vistas para análisis se construirán en mart.

7.17. Checklist de validación

Antes de continuar verificar:

- ☐ Archivo macrofinanzas.duckdb existe.
- ☐ Schema staging creado.
- ☐ Schema core creado.
- ☐ Schema mart creado.
- ☐ Tabla `tipocambiorawcreada`.
- ☐ Tabla `balancebancariorawcreada`.
- ☐ Script `initdatabase.pyejecutado`.
- ☐ Consulta de monitoreo creada.

7.18. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Base DuckDB inicializada.
- Schemas creados.
- Tablas staging creadas.
- Script de inicialización.
- Capa Bronze lista para recibir datos.

7.19. Próximo hito

En el Hito 6 construiremos el proceso ETL de carga.
Aprenderemos a:

- Leer archivos CSV.
- Cargar datos hacia DuckDB.
- Limpiar la capa staging antes de cada carga.
- Automatizar el flujo de ingesta.
- Verificar conteos de registros cargados.

Capítulo 8

Hito 6: Construcción del Primer Proceso ETL

8.1. Objetivo

El objetivo de este hito es construir el primer flujo completo de carga de datos.

Por primera vez conectaremos:

```
CSV
|
V
Python
|
V
PostgreSQL
```

Al finalizar este hito el estudiante será capaz de:

- Leer archivos CSV mediante Python.
- Conectarse a PostgreSQL utilizando SQLAlchemy.
- Cargar información en la capa staging.
- Automatizar el proceso de carga.
- Validar que los datos fueron cargados correctamente.

8.2. Resultado esperado

Al finalizar este hito los datos contenidos en:

`data/raw/tipo_cambio.csv`

`data/raw/balances.csv`

deberán encontrarse cargados en:

`staging.tipo_cambio_raw`

`staging.balance_bancario_raw`

8.3. Arquitectura del proceso

El flujo que construiremos será:

CSV

|

V

Pandas

|

V

SQLAlchemy

|

V

PostgreSQL

8.4. Paso 1: Verificar archivos fuente

Confirmar la existencia de:

`data/raw/tipo_cambio.csv`

`data/raw/balances.csv`

Verificar que pueden abrirse correctamente.

8.5. Paso 2: Crear módulo de conexión

Verificar el archivo:

etl/db.py

Contenido:

```
from sqlalchemy import create_engine
from dotenv import load_dotenv

import os

load_dotenv()

HOST = os.getenv("DB_HOST")
PORT = os.getenv("DB_PORT")
DB = os.getenv("DB_NAME")
USER = os.getenv("DB_USER")
PASSWORD = os.getenv("DB_PASSWORD")

connection_string = (
    f"postgresql+psycopg2://"
    f"{USER}:{PASSWORD}@{HOST}:{PORT}/{DB}"
)

engine = create_engine(
    connection_string
)
```

8.6. Paso 3: Crear script de carga

Crear:

etl/load_staging.py

8.7. Paso 4: Importar librerías

Agregar:

```
import pandas as pd

from db import engine
```

8.8. Paso 5: Leer tipo de cambio

```
Agregar:

fx = pd.read_csv(
    "data/raw/tipo_cambio.csv"
)

print(
    f"Filas_tipo_cambio:{len(fx)}"
)
```

8.9. Paso 6: Leer balances

```
Agregar:

balances = pd.read_csv(
    "data/raw/balances.csv"
)

print(
    f"Filas_balances:{len(balances)}"
)
```

8.10. Paso 7: Agregar metadatos

```
Agregar:

fx["archivo_origen"] = (
    "tipo_cambio.csv"
)

balances["archivo_origen"] = (
    "balances.csv"
)
```

Estos metadatos permiten rastrear el origen de cada carga.

8.11. Paso 8: Limpiar staging antes de cargar

Crear:

sql/truncate_staging.sql

Contenido:

```
TRUNCATE TABLE
staging.tipo_cambio_raw;

TRUNCATE TABLE
staging.balance_bancario_raw;
```

Esta estrategia garantiza idempotencia.

8.12. Paso 9: Ejecutar truncado desde Python

Agregar:

```
from sqlalchemy import text

with engine.begin() as conn:

    '''
    conn.execute(
        text(
            """
            TRUNCATE TABLE
            staging.tipo_cambio_raw;

            TRUNCATE TABLE
            staging.balance_bancario_raw;
            """
        )
    )
    '''
```

8.13. Paso 10: Cargar tipo de cambio

Agregar:

CAPÍTULO 8. HITO 6: CONSTRUCCIÓN DEL PRIMER PROCESO ETL73

```
fx.to_sql(  
    "tipo_cambio_raw",  
    con=engine,  
    schema="staging",  
    if_exists="append",  
    index=False  
)
```

8.14. Paso 11: Cargar balances

Agregar:

```
balances.to_sql(  
    "balance_bancario_raw",  
    con=engine,  
    schema="staging",  
    if_exists="append",  
    index=False  
)
```

8.15. Paso 12: Confirmar carga

Agregar:

```
print(  
    "Carga_ completada."  
)
```

8.16. Versión completa del script

El archivo completo debe verse así:

```
import pandas as pd  
  
from sqlalchemy import text  
  
from db import engine  
  
fx = pd.read_csv(  

```

CAPÍTULO 8. HITO 6: CONSTRUCCIÓN DEL PRIMER PROCESO ETL74

```
"data/raw/tipo_cambio.csv"
)

balances = pd.read_csv(
"data/raw/balances.csv"
)

fx["archivo_origen"] = (
"tipo_cambio.csv"
)

balances["archivo_origen"] = (
"balances.csv"
)

with engine.begin() as conn:

    '''
    conn.execute(
        text(
            """
            TRUNCATE TABLE
            staging.tipo_cambio_raw;

            TRUNCATE TABLE
            staging.balance_bancario_raw;
            """
        )
    )

    '''

fx.to_sql(
    "tipo_cambio_raw",
    con=engine,
    schema="staging",
    if_exists="append",
    index=False
)

balances.to_sql(
```

CAPÍTULO 8. HITO 6: CONSTRUCCIÓN DEL PRIMER PROCESO ETL

```
"balance_bancario_raw",
con=engine,
schema="staging",
if_exists="append",
index=False
)

print("Carga completada.")
```

8.17. Paso 13: Ejecutar ETL

Desde terminal:

```
python etl/load_staging.py
```

Salida esperada:

Carga completada.

8.18. Paso 14: Verificar en PostgreSQL

Consultar:

```
SELECT COUNT(*)
FROM staging.tipo_cambio_raw;
```

Consultar:

```
SELECT COUNT(*)
FROM staging.balance_bancario_raw;
```

Los conteos deben coincidir con los CSV.

8.19. Paso 15: Inspeccionar datos

Consultar:

```
SELECT *
FROM staging.tipo_cambio_raw
LIMIT 10;
```

Consultar:

CAPÍTULO 8. HITO 6: CONSTRUCCIÓN DEL PRIMER PROCESO ETL76

```
SELECT *
FROM staging.balance_bancario_raw
LIMIT 10;
```

8.20. Paso 16: Crear script de validación

Crear:

sql/check_load.sql

Contenido:

```
SELECT
COUNT(*) AS total_fx
FROM staging.tipo_cambio_raw;

SELECT
COUNT(*) AS total_balances
FROM staging.balance_bancario_raw;
```

8.21. Paso 17: Crear orquestador

Crear:

etl/run_pipeline.py

Contenido inicial:

```
import subprocess

subprocess.run(
[
"python",
"etl/load_staging.py"
]
)
```

Este archivo será el punto de entrada de todo el pipeline.

8.22. Buenas prácticas

Durante este proyecto seguiremos las siguientes reglas:

1. El ETL debe ser reproducible.
2. La carga debe ser idempotente.
3. No modificar datos manualmente.
4. Validar cada carga.
5. Mantener trazabilidad mediante metadatos.

8.23. Checklist de validación

Antes de continuar verificar:

- ☐ Python puede leer ambos CSV.
- ☐ Python puede conectarse a PostgreSQL.
- ☐ Staging se limpia correctamente.
- ☐ Los datos son cargados.
- ☐ Los conteos coinciden.
- ☐ El script puede ejecutarse múltiples veces.

8.24. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Script de carga funcional.
- Datos cargados en staging.
- Validaciones básicas.
- Pipeline reproducible.
- Primer flujo ETL completo.

8.25. Próximo hito

En el Hito 7 construiremos la capa Core.
Crearemos:

- Dimensión Tiempo.
- Dimensión Moneda.
- Dimensión Entidad Financiera.
- Claves subrogadas.
- Primera transformación desde staging hacia el modelo dimensional.